Metropolitan University
Department of Software Engineering

Assignment

Course Title: PROJECT ON JAVA GUI DEVELOPMENT
Course Code: SWE-282: JGD
Topic: SRS of KAAFI-app

Submitted to:
Fuad Ahmed
Professor and Head
Department of Software Engineering

Submitted by:
Name:Mahbuba Siddika Aysha
ID: 241-134-001
Name:Syed Tanjim Ahmed
ID: 231-134-002
Name:Fuad Hasan Fahim
ID: 241-134-018
Department: Software Engineering
Batch: 6th

Date of Submission: 26th January 2026

TABLE OF CONTENTS

1.0 EXECUTIVE SUMMARY

1.1 Project Vision & Business Value Proposition

1.2 Current State: MVP Validation

1.3 Strategic Roadmap: Four-Phase Evolution

1.4 Key Success Metrics & KPIs

2.0 INTRODUCTION

2.1 Purpose of This Document

2.2 Document Scope & Evolution

2.2.1 In-Scope Features by Phase

2.2.2 Out-of-Scope Items

2.3 Definitions, Acronyms & Abbreviations

2.4 Document Conventions & References

3.0 PRODUCT DESCRIPTION & MVP BASELINE

3.1 Product Perspective & Market Context

3.2 User Classes, Personas & Characteristics

3.2.1 Learner

3.2.2 Instructor

3.2.3 Administrator

3.3 Current Implementation: MVP Baseline

3.3.1 MVP Architecture & Technology Stack

3.3.2 Validated Core Features

3.3.3 Design Decisions & Trade-offs

3.4 Operating Environment & Constraints

4.0 PROJECT METHODOLOGY & DEVELOPMENT ROADMAP

4.1 Hybrid SDLC Model Justification

4.1.1 Phase 1ˇ Agile MVP Development

4.1.2 Phase 24Structured-Iterative Hybrid Approach

4.2 Detailed Four-Phase Implementation Roadmap

4.2.1 Phase 1ˇ MVP Core Platform __Complete)

4.2.2 Phase 2ˇ Scaling & Hardening __Active)

4.2.3 Phase 3ˇ Advanced Integration __Planned)

4.2.4 Phase 4ˇ Enterprise & Optimization __Future)

4.3 Use Case Models & Key User Stories

5.0 REQUIREMENTS SPECIFICATION

5.1 Functional Requirements

5.1.1 User Management Module __FUNUM+ <MVP/PHASE2

5.1.2 Course Marketplace Module __FUNCM+ <MVP/PHASE2

5.1.3 Learning Environment Module __FUNLE+ <MVP/PHASE2

5.1.4 Payment & Cryptocurrency Module __FUNPT/CP+ <PHASE3

5.1.5 Blockchain & Cryptography Module __FUNBC+ <PHASE3

5.1.6 Data Analytics & BI Module __FUNDA+ <PHASE2/PHASE4

5.1.7 AI/ML Features Module __FUNAI+ <PHASE3

5.1.8 Administration Module __FUNAD+ <MVP/PHASE2

5.2 Non-Functional Requirements

5.2.1 Performance Requirements

5.2.2 Security & Privacy Requirements

5.2.3 Usability & Accessibility Requirements

5.2.4 Reliability, Availability & Scalability

5.2.5 Compliance & Legal Requirements

6.0 SYSTEM ARCHITECTURE & DESIGN

6.1 High-Level Architecture Evolution

6.1.1 MVP Architecture

6.1.2 Target Architecture

6.2 Technology Stack Evolution

6.2.1 MVP Technology Stack

6.2.2 Target Technology Stack

6.3 Detailed Component Design

6.3.1 Frontend Architecture __Flutter)

6.3.2 Backend Services Design

6.3.3 Data Architecture & Database ERD

6.4 Advanced Modules Architecture

6.4.1 Blockchain Integration Architecture
6.4.2 WebAssembly Module Architecture
6.4.3 MLOps & AI Pipeline Architecture
7.0 EXTERNAL INTERFACES
7.1 User Interface Specifications
7.1.1 Current MVP UI
7.1.2 Target Design System
7.2 Hardware & Software Interfaces
7.3 Communication Interfaces
7.3.1 API Specifications
7.3.2 Third-Party Service Integrations
8.0 TESTING, DEPLOYMENT & OPERATIONS
8.1 Comprehensive Testing Strategy
8.1.1 Test Levels & Coverage
8.1.2 Specialized Testing
8.2 DevOps & MLOps Implementation
8.2.1 CI/CD Pipeline Design
8.2.2 Infrastructure as Code
8.3 Deployment & Monitoring Strategy
9.0 RISK ASSESSMENT & MITIGATION
9.1 Technical Risks
9.2 Business & Market Risks
9.3 Project Management Risks
9.4 Risk Mitigation Strategies
10.0 PROJECT MANAGEMENT
10.1 Milestones & Timeline
10.2 Resource Allocation
10.3 Success Criteria & Acceptance Plan
11.0 CONCLUSION
12.0 REFERENCES
APPENDICES

1.0 EXECUTIVE SUMMARY

1.1 Project Vision & Business Value Proposition

KAAFI is a mobile-first, two-sided marketplace dedicated to democratizing practical skill acquisition in Bangladesh. Our mission is to bridge the critical gap between theoretical knowledge and real-world application by connecting verified experts with learners through focused, affordable micro-courses.

Core Value Proposition: Unlike traditional e-learning platforms, KAAFI enforces a **quality-first, verification-driven model**. We guarantee that every course is taught by a vetted practitioner, focuses on an immediately applicable micro-skill, and is priced accessibly (under 1000 BDT). This creates a trusted ecosystem where learners gain tangible skills, and experts can monetize their hands-on knowledge effectively.

1.2 Current State: MVP Validation

We have successfully developed and launched a **Minimum Viable Product (MVP)**, validating our core technical approach and user hypotheses. The MVP is a fully functional application built with **Flutter, GetX, and Supabase**, demonstrating key user journeys:

- User registration and authentication.
- Instructor course creation and publishing.
- Learner discovery, enrollment, and basic video-based learning.

This working foundation proves the technical feasibility of our platform and provides a critical feedback loop for guiding all future development phases.

1.3 Strategic Roadmap: Four-Phase Evolution

KAAFI's development is strategically planned as a four-phase evolution to manage risk, validate growth, and integrate advanced technologies systematically.

Phase	Name	Status	Primary Objective
1	**MVP Core**	✅ **COMPLETE**	Validate core concept & user flow with a functional product.

Phase	Name	Status	Primary Objective
2	**Scaling & Hardening**	🔄 **IN PROGRESS**	Transform MVP into a scalable, secure, and polished product.
3	**Advanced Integration**	📋 **PLANNED**	Integrate AI, Blockchain, and Cryptocurrency as key differentiators.
4	**Enterprise & Optimization**	🔮 **FUTURE**	Evolve into a data-driven, community-governed learning ecosystem.

1.4 Key Success Metrics & KPIs

We will measure the success and impact of the KAAFI platform through the following key metrics, aligned with each phase of our roadmap:

Metric Category	Key Performance Indicator (KPI)	Phase 1–2 Target	Phase 3–4 Target
User Adoption	Monthly Active Users (MAU)	1,000	10,000+
Learning Effectiveness	Course Completion Rate	60%	75%+
Marketplace Health	Instructor Verified & Active	50	500+
Business Sustainability	Average Revenue Per User (ARPU)	150 BDT	250 BDT
Platform Quality	System Uptime (Reliability)	99.5%	99.9%
User Satisfaction	Net Promoter Score (NPS)	>20	>40

2.0 INTRODUCTION

2.1 Purpose of This Document

This Software Requirements Specification (SRS) document serves as the definitive source of truth for the KAAFI platform. Its primary purpose is to:

- **Define the Complete System:** Provide a detailed, unambiguous specification of all functional and non-functional requirements, from the initial Minimum Viable Product (MVP) through its evolution into an enterprise-scale application.
- **Guide Development & Validation:** Serve as the principal reference for software developers, system architects, testers, and DevOps/MLOps engineers to guide implementation, create test plans, and validate deliverables.
- **Communicate Scope & Vision:** Articulate the business context, strategic phased roadmap, and selected development methodology to all stakeholders, ensuring a shared understanding of the project's goals, constraints, and evolution.
- **Establish a Living Baseline:** Function as a controlled, versioned document that evolves with the project, recording decisions and providing a stable foundation for all future development phases.

2.2 Document Scope & Evolution

This SRS defines the scope for the KAAFI project across its planned four-phase evolution. The scope is dynamic, expanding with each phase as features are validated and new capabilities are integrated.

2.2.1 In-Scope Features by Phase

Phase	Name & Goal	Methodology	Scopes
1	**MVP Core __Completed)**Validate demand & core tech.	**Agile**	Functional app, core user flows, MVC Architecture codebase, Supabase validation and database usage.
2	**Platform Hardening & Integration**Secure, polish, and integrate **critical business systems**.	**Hybrid**(**Spiral** for payments, **Agile** for UI/Infra)	1. **Security & DevOps:** Audit, CI/CD, IaC.2. **Payment System:** Bkash/Nagad integration.3. **UI/UX**ˇ Design system & polish.4. **Analytics:** Basic dashboard.
3	**Advanced Technology & Scaling** Integrate **high-risk, high-innovation** features to differentiate.	**Spiral**__Risk-driven for blockchain/AI¯¯	1. **Blockchain & Crypto:** NFT certificates, crypto payments.2. **AI/ML**ˇ personalization and recommendation engine.3. **Performance:** WASM modules.
4	**Enterprise Intelligence & Evolution**Become a data-driven, self-improving platform.	**Agile-DevOps**	1. **Predictive BI & MLOps.** 2. **Community Governance __DAO¯¯.** 3. **Automated feedback loops.4.Make it sustainble .**

2.2.2 Out-of-Scope Items

The following items are explicitly excluded from the KAAFI project scope across all phases:

- Development of custom hardware.
- Full offline functionality for video streaming.
- Direct accreditation of courses by government education boards.

2.3 Definitions, Acronyms & Abbreviations

Term	Definition
MVP	Minimum Viable Product. The initial working version of KAAFI with core features to validate the market need.
SDLC	Software Development Life Cycle. The structured process for software development __Hybrid model used here).
BaaS	Backend-as-a-Service. Cloud service (e.g., Supabase) providing database, authentication, and APIs without server management.

Term	Definition
DAO	Decentralized Autonomous Organization. A community-governed entity managed by blockchain smart contracts.
NFT	Non-Fungible Token. A unique digital certificate on the blockchain, used for verifiable course completion credentials.
WASM	WebAssembly. A high-performance, portable binary format for executing code on the web and mobile devices.
SRS	Software Requirements Specification. __This document).

2.4 Document Conventions & References

2.4.1 Conventions

- **Requirement Tags:** Functional requirements are tagged with their implementation phase: `+MVP<`, `+PHASE2<`, `+PHASE3<`, `+PHASE4<`.
- **Priority Levels:** In requirements tables, priorities are denoted as: **MUST** __Critical), **SHOULD** __Important), **COULD** __Nice-to-have).
- **Cross-Referencing:** References to other sections are indicated as `__Section X.Y`.

2.4.2 References

- IEEE Std 8301998 *Recommended Practice for Software Requirements Specifications*.
- Project Lab II Guidelines, Institute of Information Technology, University of Dhaka.
- Flutter Documentation: https://flutter.dev
- Supabase Documentation: https://supabase.com/docs

3.0 PRODUCT DESCRIPTION & MVP BASELINE

3.1 Product Perspective & Market Context

KAAFI is a project, two-sided marketplace platform designed to address the practical skills gap in Bangladesh. It connects verified experts who create focused, immediately applicable micro-courses with learners seeking affordable, practical knowledge.

The current **Minimum Viable Product __MVP** relies on Supabase as its backend, and while the project is educational in scope, numerous advanced features remain to be integrated throughout its development until the final year project submission. KAAFI aims to establish a new category of learning that is solely focused on raising awareness about one's surroundings and enhancing daily life skills through accessible education.

3.2 User Classes, Personas & Characteristics

3.2.1 Learner

- **Description:** The primary consumer, typically a student, young professional, or hobbyist seeking to acquire a specific, practical skill.
- **Key Goals:** Find high-quality, affordable courses __¯ 1000 BDT¯¯; learn at their own pace via smartphone; use trusted local payment methods; receive verifiable proof of skill acquisition.
- **Technical Proficiency:** Comfortable with smartphone apps and digital payments.

3.2.2 Instructor

- **Description:** A domain expert or practitioner (e.g., electrician, graphic designer, gardener) who wants to monetize their practical knowledge.
- **Key Goals:** Easily create and publish courses; reach a large audience of motivated learners; earn supplemental income; build a reputation as a verified expert.
- **Technical Proficiency:** Requires an intuitive course creation tool; may not be highly tech-savvy.

3.2.3 Administrator

- **Description:** Platform operator responsible for system health, user safety, and content quality.
- **Key Goals:** Manage user accounts and permissions; review and verify instructor applications; moderate course content; monitor platform analytics and financial transactions.
- **Technical Proficiency:** High; comfortable with dashboards and operational tools.

3.3 Current Implementation: MVP Baseline

A fully functional Minimum Viable Product __MVP¯¯ has been developed , serving as the foundation for all future development.

3.3.1 MVP Architecture & Technology Stack

- **Frontend:** Built with **Flutter** __Dart) for cross-platform deployment. Uses the **GetX** framework for state management, routing, and dependency injection, chosen for its productivity and low boilerplate and software engineering principle.
- **Backend: Supabase** serves as the Backend-as-a-Service __BaaS¯¯, providing:
  - **PostgreSQL Database:** For all structured data (users, courses, enrollments).
  - **Authentication:** Row-Level Security __RLS¯¯ for user management.
  - **Storage:** Buckets for course video and resource files.
  - **Realtime Listeners:** For live updates.
- **Architecture Pattern:** The codebase follow MVC architecture adhering to **SOLID principles** to ensure maintainability, testability, and clear separation of concerns.

3.3.2 Validated Core Features

The MVP has successfully implemented and validated the following core user journeys:

- User registration and login via email/password with Supabase Auth.
- Instructor profile creation and a basic course publishing flow.
- A learner-facing course catalog with search functionality.
- Course enrollment and integration of a basic video player for content consumption.
- A simulated payment flow, with integration points ready for Bkash/Nagad APIs.

3.3.3 Design Decisions & Trade-offs

- **GetX for Speed:** Chosen over Bloc or Riverpod for the MVP due to its rapid development cycle, allowing faster validation of core business logic.
- **Supabase over Custom Backend:** Eliminated the need for initial DevOps, database administration, and auth server development, compressing months of backend work into weeks.
- **"Functional over Perfect" UI** Deliberate trade-off to prioritize a complete, working user flow over pixel-perfect UI design in the first iteration, aligning with the Lean Startup "build-measure-learn" loop.

3.4 Operating Environment & Constraints

- **Software Environment:**
  - **Client:** Android API 24, iOS 14.0, modern web browsers.
  - **Server:** Deployed on Supabase cloud platform. Future phases will migrate to a dedicated cloud provider AWS/GCP.
- **Hardware Environment:** Standard smartphones with internet connectivity 3G minimum) and pc or computer for instructor.
- **Key Constraints:**
  - **Technical:** The initial architecture is bound to Supabase's platform limits, necessitating a planned migration for high-scale phases.
  - **Legal:** All platform content must comply with Bangladesh's Digital Security Act and copyright laws.

4.0 PROJECT METHODOLOGY & DEVELOPMENT ROADMAP

4.1 Hybrid SDLC Model Justification

The KAAFI project does not adhere to a single, rigid Software Development Life Cycle SDLC model. Instead, it employs a **pragmatic Hybrid SDLC** where the methodology is chosen per phase based on the phase's primary goal, risk profile, and need for feedback. This adaptive approach is

essential for a project that transitions from validating a novel concept to implementing complex, high-stakes technologies.

4.1.1 Phase 1: Agile MVP Development

The initial MVP phase followed a lightweight **Agile/Iterative** methodology.

- **Reasoning:** The goal was **speed and validation** under high uncertainty. Building a working product quickly to test core assumptions with real users was paramount.
- **Execution:** Development occurred in two-week sprints focusing on end-to-end user stories (e.g., "As a Learner, I can browse and enroll in a course"). Daily stand-ups and a flexible backlog allowed for rapid adaptation based on technical hurdles and early user feedback.

4.1.2 Phase 2-4: Structured-Iterative Hybrid Approach

For all subsequent phases, the project adopts a **Hybrid model combining Spiral and Agile** elements.

- **Reasoning:** The goals shift to **scalability, security, and integrating complex technologies** __AI, Blockchain). These carry significant technical and regulatory risks that must be proactively managed before full-scale development.
- **Execution:** Each phase will begin with a **Spiral Model** cycle: identifying major risks (e.g., "Blockchain gas fees are too high," "AI model is biased"), prototyping solutions, and evaluating outcomes. Once key risks are mitigated, development proceeds in **Agile Sprints** to build out features within the now-defined safe boundaries. This ensures robustness where needed while maintaining development flexibility.

4.2 Detailed Four-Phase Implementation Roadmap

Phase	Name & Goal	Methodology	Key Deliverables / Focus __Refined)
1	**MVP Core __Complete)**Validate demand & core tech.	**Agile**	Functional app, core user flows, MVC Architecture codebase, Supabase validation and database usage.
2	**Platform Hardening & Integration**Secure, polish, and integrate **critical business systems**.	**Hybrid**(**Spiral** for payments, **Agile** for UI/Infra)	1. **Security & DevOps:** Audit, CI/CD, IaC.2. **Payment System:** Bkash/Nagad integration.3. **UI/UX**ˇ Design system & polish.4. **Analytics:** Basic dashboard.
3	**Advanced Technology & Scaling**Integrate **high-risk, high-innovation** features to differentiate.	**Spiral**__Risk-driven for blockchain/AI¯¯	1. **Blockchain & Crypto:** NFT certificates, crypto payments.2. **AI/ML**ˇ personalization and recommendation engine.3. **Performance:** WASM modules.

Phase	Name & Goal	Methodology	Key Deliverables / Focus __Refined)
4	**Enterprise Intelligence & Evolution**Become a data-driven, self-improving platform.	**Agile-DevOps**	1. **Predictive BI & MLOps.** 2. **Community Governance __DAO__.** 3. **Automated feedback loops.4.Make it sustainble .**

4.3 Use Case Models & Key User Stories

This section summarizes the primary interactions within the KAAFI system. Detailed use case specifications are provided in **Appendix A**.

- **Primary Actor: Learner**
  - **UCL1Discover Courses.** Browse, search, and filter the marketplace to find relevant micro-courses.
  - **UCL2Acquire Course.** Enroll in a course by completing a secure payment via integrated gateways.
  - **UCL3Consume Content.** Watch lesson videos, track progress, and take notes within the learning environment.
  - **UCL4Receive Certification.** Obtain a verifiable digital certificate __NFT__ upon successful course completion.
- **Primary Actor: Instructor**
  - **UCI1Get Verified.** Submit credentials and complete the platform's verification process.
  - **UCI2Create Content.** Use the course builder to structure lessons, upload videos, and set pricing.
  - **UCI3Manage Earnings.** View analytics on course sales, student engagement, and revenue.
- **Primary Actor: Administrator**
  - **UCA1Moderate Platform.** Manage user accounts, review instructor applications, and audit course content.
  - **UCA2Monitor Health.** Oversee system performance, transaction integrity, and platform analytics via the admin dashboard.
- **Supporting Actor: System __Automated)**
  - **UCS1Personalize Experience.** Recommend courses to learners based on behavior and profile .
  - **UCS2Mint Certificate.** Execute smart contract calls to issue blockchain-based certificates upon completion triggers.

5.0 REQUIREMENTS SPECIFICATION

5.1 Functional Requirements

Requirements are categorized by system module and tagged with their target implementation phase (`+MVP<`, `+PHASE2<`, `+PHASE3<`, `+PHASE4<`). **Priority:** MUST __Critical), SHOULD __Important), COULD (Optional).

5.1.1 User Management Module (FUN-UM)

ID	Requirement Description	Priority	Phase
FUN□UM□001	A user shall be able to register using an email address and a mobile OTP for verification.	MUST	+MVP<
FUN□UM□002	A registered user shall be able to log in using their email and password.	MUST	+MVP<
FUN□UM□003	A user shall have a public profile displaying their name, photo, and bio.	SHOULD	+MVP<
FUN□UM□004	An aspiring instructor shall submit documents (e.g., NID, portfolio) for manual verification by an admin.	MUST	+PHASE2<
FUN□UM□005	Verified instructors shall have a distinct badge displayed on their profile and courses.	MUST	+PHASE2<
FUN□UM□006	Users shall be able to enable Two-Factor Authentication __2FA¯¯ for their account.	COULD	+PHASE3<

5.1.2 Course Marketplace Module (FUN-CM)

ID	Requirement Description	Priority	Phase
FUN□CM□001	Learners shall be able to browse all published courses in a list or grid view.	MUST	+MVP<
FUN□CM□002	The system shall allow filtering courses by category, price range __0¯5¯00, 501□1000 BDT□ , and rating.	MUST	+MVP<
FUN□CM□003	Learners shall be able to search for courses by title, instructor name, or skill keyword.	MUST	+MVP<
FUN□CM□004	Learners shall be able to save courses to a "Wishlist" for later viewing.	SHOULD	+PHASE2<
FUN□CM□005	The system shall display a "Recommended For You" section based on the learner's browsing and enrollment history.	SHOULD	+PHASE3<
FUN□CM□006	Each course listing shall display a preview of the first lesson (max 3 minutes).	COULD	+PHASE2<

5.1.3 Learning Environment Module (FUN-LE)

ID	Requirement Description	Priority	Phase
FUN□LE□001	The system shall provide a video player capable of streaming uploaded lesson content.	MUST	+MVP<
FUN□LE□002	The video player shall automatically save the playback position and allow the user to resume from that point.	MUST	+MVP<
FUN□LE□003	Learners shall be able to mark a lesson as complete, with progress visually tracked for the entire course.	MUST	+MVP<
FUN□LE□004	Learners shall be able to take timestamped notes while watching a video lesson.	SHOULD	+PHASE2<
FUN□LE□005	Upon 100% course completion, the system shall award a completion certificate (digital PDF⌒.	MUST	+PHASE2<
FUN□LE□006	The video player shall support adaptive bitrate streaming, adjusting quality based on network speed.	SHOULD	+PHASE3<

5.1.4 Payment & Cryptocurrency Module (FUN-PT/CP)

ID	Requirement Description	Priority	Phase
FUN□PT□001	The system shall integrate with the Bkash payment gateway for processing course payments in BDT.	MUST	+MVP<
FUN□PT□002	The system shall integrate with the Nagad payment gateway as an alternative payment method.	SHOULD	+PHASE2<
FUN□PT□003	Users shall be able to view a detailed transaction history for all their payments.	MUST	+PHASE2<
FUN□CP□001	The system shall integrate a cryptocurrency wallet (e.g., MetaMask) to allow payments in USDT.	SHOULD	+PHASE3<
FUN□CP□002	The system shall display real-time exchange rates for cryptocurrency payments.	MUST	+PHASE3<
FUN□CP□003	Users shall have the option to automatically convert crypto payments to BDT at the time of transaction.	COULD	+PHASE3<

5.1.5 Blockchain & Cryptography Module (FUN-BC)

ID	Requirement Description	Priority	Phase
FUN□BC□001	Upon course completion, the system shall mint a unique Non-Fungible Token __NFT⌒ certificate on a blockchain (e.g., Polygon) for the learner.	MUST	+PHASE3<
FUN□BC□002	The NFT certificate shall contain metadata: learner name, course title, completion date, and a link to the issuing platform.	MUST	+PHASE3<
FUN□BC□003	A public verification portal shall allow anyone to verify the authenticity of a certificate by entering its transaction hash or contract address.	SHOULD	+PHASE3<

ID	Requirement Description	Priority	Phase
FUN□BC□004	All sensitive API keys and wallet mnemonics shall be encrypted using AES□256 and stored securely.	MUST	+PHASE3<
FUN□BC□005	User-to-user messages within the platform shall be end-to-end encrypted.	COULD	+PHASE4<

5.1.6 Data Analytics & BI Module (FUN-DA)

ID	Requirement Description	Priority	Phase
FUN□DA□001	Instructors shall have a dashboard showing course views, enrollments, completion rates, and revenue.	MUST	+PHASE2<
FUN□DA□002	Administrators shall have a dashboard showing platform KPIs: total users, active courses, monthly revenue, and geographic distribution.	MUST	+PHASE2<
FUN□DA□003	The system shall generate weekly automated summary reports for instructors via email.	SHOULD	+PHASE2<
FUN□DA□004	The admin dashboard shall predict course demand trends using historical enrollment data.	COULD	+PHASE4<
FUN□DA□005	The system shall use NLP to analyze course review sentiment and highlight common themes for instructors.	COULD	+PHASE4<

5.1.7 AI/ML Features Module (FUN-AI)

ID	Requirement Description	Priority	Phase
FUN□AI□001	An AI-powered chatbot shall answer learner questions related to the course content they are currently taking.	SHOULD	+PHASE3<
FUN□AI□002	The system shall recommend courses to learners using a collaborative filtering algorithm based on similar users' behavior.	SHOULD	+PHASE3<
FUN□AI□003	The system shall analyze video transcripts to automatically generate keywords and improve searchability.	COULD	+PHASE3<
FUN□AI□004	The system shall flag potentially inappropriate or low-quality content for admin review using text and metadata analysis.	COULD	+PHASE4<

5.1.8 Administration Module (FUN-AD)

ID	Requirement Description	Priority	Phase
FUN□AD□001	Admins shall be able to view, suspend, or delete any user account.	MUST	+MVP<

ID	Requirement Description	Priority	Phase
FUN□AD□002	Admins shall have a queue to review and approve/reject instructor verification applications.	MUST	+PHASE2<
FUN□AD□003	Admins shall be able to review and take down reported courses or user content.	MUST	+PHASE2<
FUN□AD□004	Admins shall be able to configure platform-wide settings, such as commission rates and featured courses.	SHOULD	+PHASE2<
FUN□AD□005	Admins shall have access to audit logs for all critical actions performed on the platform.	MUST	+PHASE3<

5.1.9 Algorithmic Core Module (FUN-ALGO)

This module encompasses the design and implementation of KAAFI's foundational numerical library and the applied algorithms built upon it for search, personalization, and data efficiency.

ID	Requirement Description	Priority	Phase	Category & Purpose
FUN□ALGO□001	**Numerical Foundation Library:** Implement a core library with stable **linear algebra operations** (matrix multiplication, SVD¯¯ and **statistical primitives** (probability distributions).	MUST	+PHASE3<	**FOUNDATION**ˇ The bedrock for all custom algorithms.
FUN□ALGO□002	**Search & Ranking Engine:** Implement **TF□IDF for relevance ranking** and **Levenshtein Distance for typo tolerance** to power the course search.	MUST	+PHASE2<	**APPLICATION (Search):** Directly improves user discovery.
FUN□ALGO□003	**Personalized Recommendation Engine:** Build a hybrid system using **Cosine Similarity (content-based)** and **Collaborative Filtering**, leveraging the foundation library __FUN□ALGO□001□ .	SHOULD	+PHASE3<	**APPLICATION (Personalization):** Drives engagement.
FUN□ALGO□004	**Gradient-Based Optimization:** Implement optimizers __SGD¯¯ by manually calculating	SHOULD	+PHASE3<	**FOUNDATION**ˇ Enables learning in custom models.

ID	Requirement Description	Priority	Phase	Category & Purpose
	derivatives for model training.			
FUN□ALGO□005	**Efficiency & Caching Algorithms:** Implement **LRU Caching** for frequent data and **Reservoir Sampling** for random selections to optimize performance.	SHOULD	+PHASE2<	**APPLICATION (Performance):** Makes the platform fast and scalable.
FUN□ALGO□006	**Hybrid Intelligence Layer:** The production system shall combine predictions from custom algorithms __FUN□ALGO□003__ with third-party AI APIs (e.g., Mistral AI__ using a custom fusion logic.	MUST	+PHASE3<	**INTEGRATION**ˇ Shows practical use of your core.

5.2 Non-Functional Requirements

5.2.1 Performance Requirements

- **NFRPERF□001□** The application's home screen shall load within **2 seconds** for 95% of users on a 4G connection.
- **NFRPERF□002□** Search results for courses shall be returned in less than **1 second** for a catalog of up to 10,000 courses.
- **NFRPERF□003□** Video playback shall start streaming within **3 seconds** on a 3G __512 Kbps) network.
- **NFRPERF□004□** The system shall support up to **1,000 concurrent users** without significant degradation in response time.

5.2.2 Security & Privacy Requirements

- **NFRSEC□001□** All user passwords shall be hashed using **bcrypt** with a work factor of 12 before storage.
- **NFRSEC□002□** All data transmission between client and server shall be encrypted using **TLS 1.3**.
- **NFRSEC□003□** The system shall be scanned for OWASP Top 10 vulnerabilities (e.g., SQL Injection, XSS__ quarterly.
- **NFRSEC□004□** User personal data shall not be shared with third parties without explicit consent, adhering to data minimization principles.

5.2.3 Usability & Accessibility Requirements

- **NFRUSA□001□** A new user shall be able to complete registration and enroll in their first course within **4 minutes**.
- **NFRUSA□002□** The user interface shall comply with **WCAG 2.1 Level AA** accessibility standards (e.g., sufficient color contrast, screen reader support).
- **NFRUSA□003□** The application shall be fully navigable and functional using only a keyboard (web version).

5.2.4 Reliability, Availability & Scalability

- **NFRREL□001□** The core platform shall maintain an uptime of **99.5%** during business hours □6ˇ00 AM ˇ12¯00 AM BDT□ .
- **NFRREL□002□** Automated database backups shall be performed daily and retained for 30 days.
- **NFRSCAL□001□** The database schema shall be designed to support a tenfold (10x) increase in users and courses without requiring a complete redesign.
- **NFRSCAL□002□** The application servers shall be capable of horizontal scaling to handle traffic spikes.

5.2.5 Compliance & Legal Requirements

- **NFRCOMP□001□** The payment processing module shall adhere to the security standards of integrated gateways □Bkash/Nagad) and follow **PCI□DSS** guidelines for any card data handling.
- **NFRCOMP□002□** The platform's terms of service, privacy policy, and data handling practices shall comply with relevant Bangladeshi laws, including the **Digital Security Act** and data protection guidelines.

6.0 SYSTEM ARCHITECTURE & DESIGN

6.1 High-Level Architecture Evolution

6.1.1 MVP Architecture

The MVP follows an **MVC □Model-View-Controller) architectural pattern** with a centralized backend. It implements a three-tier data strategy to optimize performance and user experience:

- **Server Database □Supabase PostgreSQL¯˘** The primary source of truth for all structured data.
- **Client Caching:** In-memory caching of frequently accessed data (e.g., course listings, user profiles) to reduce network calls.
- **Local Database □SQLite):** Persistent storage for offline access to user-specific data such as course progress and notes.

The system employs an **intelligent data-fetching strategy**:

- Initial app load retrieves a limited dataset (e.g., 100 featured courses, promotional banners, and marketing content).
- Additional data is fetched on-demand via pagination or user-triggered actions (e.g., scrolling, searching).
- Background synchronization ensures cached and local data remain consistent with the server.

Key Characteristics:

- **Client (Flutter App):** Contains all presentation logic, UI components, and state management using GetX. It communicates directly with Supabase via auto-generated APIs.
- **Server (Supabase BaaS):** A unified, managed backend layer providing:
  - **PostgreSQL Database** with Row-Level Security (RLS) policies.
  - **Authentication & Authorization** services.
  - **Auto-generated REST & GraphQL APIs**.
  - **Realtime Subscriptions** for live updates (e.g., new course notifications).
  - **Storage Buckets** for course media (videos, thumbnails, resources).

6.1.2 Target Architecture (Phase 3+)

The target architecture evolves into a **decoupled microservices model** to support scalability, independent deployment of advanced features, and fault isolation. An API Gateway acts as the single entry point, routing requests to specialized services.

Key Characteristics:

- **API Gateway:** Handles request routing, rate limiting, authentication (JWT verification), and logging. Decouples clients from internal services.
- **Microservices:** Loosely coupled, independently deployable services, each owning its domain logic and database:
  - **User Service:** Manages profiles, authentication, and instructor verification.
  - **Course Service:** Handles course CRUD, catalog, and search (integrated with Elasticsearch).
  - **Enrollment & Learning Service:** Manages student progress, video streaming metadata, and certificate issuance.
  - **Payment Service:** Orchestrates transactions across Bkash, Nagad, and cryptocurrency gateways.
  - **AI Service:** Hosts ML models for recommendations, chatbots, and analytics.
  - **Notification Service:** Sends emails, SMS, and push notifications.

- **Message Broker __Redis):** Facilitates asynchronous, event-driven communication between services (e.g., "PaymentCompleted" event triggers "EnrollmentConfirmed").
- **Blockchain Node/Service:** A dedicated service interacts with blockchain networks (e.g., Polygon) to mint NFTs and verify transactions.

6.2 Technology Stack Evolution

6.2.1 MVP Technology Stack

Layer	Technology	Justification for MVP
Frontend	Flutter __Dart), GetX	Rapid cross-platform development; GetX reduces boilerplate for state & navigation.
Backend __BaaS__	Supabase __PostgreSQL, Auth, Storage)	Eliminates backend DevOps; provides instant, secure APIs and realtime features.
Payments	Didnt added	null
State Management	GetX	Integrated solution for state, routes, dependencies; low learning curve.
Architecture	MVC	Ensures maintainable, testable code from the start, easing future refactoring.

6.2.2 Target Technology Stack

Layer	Technology	Justification for Evolution
Frontend	Flutter, Bloc	Riverpod offers more predictable, testable, and scalable state management for complex apps.
Backend __Microservices)	**FastAPI __Python)**, PostgreSQL, **Redis**	FastAPI enables high-performance, async microservices. Redis for caching & messaging.
API Gateway	**Nginx** or **Kong**	Industry-standard for routing, load balancing, and security at the edge.
Container & Orchestration	**Docker**, **Kubernetes (K8s)**	Enables consistent deployment, scaling, and management of microservices.
Blockchain	**Solidity**, **Hardhat**, **Alchemy/Infura(need further research to do)**	Standard stack for Ethereum/Polygon smart contract development and deployment.
AI/ML	**Python**, **PyTorch/TensorFlow**, **FastAPI**	Rich ecosystem for model development; served via dedicated microservices.
CI/CD & DevOps	**GitHub Actions**, **Terraform**	Automates testing, building, and provisioning cloud infrastructure.

6.3 Detailed Component Design

6.3.1 Frontend Architecture (Flutter)

```
lib/
├── common/              # Shared UI Components
│   ├── button/
│   ├── card/
│   ├── icon/
│   ├── image/
│   ├── text/
│   └── textfield/
├── core/                # Application Infrastructure
│   ├── bindings/        # Dependency injection setup
│   ├── Constants/       # App constants (strings, colors, API endpoints)
│   ├── route/           # Navigation/routing configuration
│   ├── Services/        # Singleton services __Auth, Storage, Network)
│   └── utils/           # Helper functions & utilities
├── data/                # Data Layer __Model)
│   ├── local/           # Local data sources __SQLite, SharedPreferences)
│   ├── models/          # Data classes __DTOs/Entities)
│   └── repository/      # Repository pattern implementations
├── exports/             # Barrel files for clean imports
│   └── data_paths.dart  # Centralized export paths
├── feature/             # Feature Modules
│   ├── ai_module/       # AI chatbot feature
│   ├── intro_screen/    # Onboarding flow
│   └── screens/         # Main application screens
├── generated/           # Auto-generated files
│   ├── assets.dart      # Asset references
│   ├── main.dart        # Entry point
│   └── navigation.dart  # Navigation setup
└── main.dart            # Primary application file
```

6.3.2 Backend Services Design (Target State)

Each microservice in the target architecture will follow a similar internal pattern:

- **API Layer:** FastAPI routers defining RESTful or GraphQL endpoints.
- **Service Layer:** Contains core business logic and application rules.
- **Data Access Layer __Repository):** Abstracts database operations (using an ORM like SQLAlchemy or asyncpg).
- **Domain Models:** Pydantic models for data validation and serialization.
- **Communication:** Services publish and subscribe to domain events (e.g., `UserRegistered`, `PaymentSucceeded`) via the Redis message broker, ensuring loose coupling.

6.3.3 Data Architecture & Database ERD

The core data model is relational, centered around a few key entities with clear relationships:

- `users`: Stores profiles for learners, instructors, and admins. Linked to auth provider (Supabase Auth UUID).
- `courses`: Owned by an instructor (`user_id`). Contains title, description, price, status.
- `enrollments`: Junction table linking a `user_id` (learner) to a `course_id`, with additional fields like `progress_percentage`.
- `lessons`: Belongs to a course (`course_id`). Contains video URL, duration, sort order.
- `transactions`: Records all financial transactions linked to a `user_id` and `course_id`.

6.4 Advanced Modules Architecture

6.4.1 Blockchain Integration Architecture

This module is designed as a dedicated **"Certificate Minting Service"** within the target microservices ecosystem.

- **Trigger:** The "Enrollment Service" emits a `CourseCompleted` event with learner and course details.
- **Processing:** The "Certificate Service" consumes the event, generates certificate metadata, and calls a pre-deployed **Smart Contract** (ERC721/1155) on the Polygon blockchain via a node provider (Alchemy/Infura).
- **Minting:** The smart contract mints a unique NFT to the learner's public wallet address (collected during sign-up).
- **Confirmation:** The service captures the transaction hash and updates the platform's database, linking the course enrollment to the on-chain certificate. The hash is displayed in the user's profile for verification.

6.4.2 WebAssembly Module Architecture

WASM modules will be used to offload performance-critical tasks from the main JavaScript/Dart thread to near-native speed.

- **Development:**
  - using **C/C++**: modules will be compiled with **Emscripten** or Clang targeting `wasm32`.
- **Integration:**
  - In the Flutter web app, `.wasm` files will be fetched and instantiated using `dart:js` interop or packages like `wasm_run`.
  - On mobile, performance-critical logic will be compiled as native FFI plugins (since WASM runtimes are not yet first-class on iOS/Android).
- **Use Cases:**

- Client-side **video thumbnail generator** (efficient frame extraction without blocking UI).
- **Complex analytics calculation engine** (personalized scoring, statistical models), ensuring UI responsiveness.

- **Notes:**
  - Optimize memory transfer between Dart and WASM using typed arrays.
  - Ensure secure loading of `.wasm` files with integrity checks.

6.4.3 MLOps & AI Pipeline Architecture

A continuous pipeline to manage the lifecycle of AI models (e.g., for recommendations).

- **Data Collection & Feature Store:** User interactions (clicks, completions) are logged and processed into consistent features stored in a **feature store** (e.g., **Feast**).
- **Model Training (Offline):** Training pipelines (using **MLflow** for tracking) are triggered on a schedule or by new data, pulling features from the store. Models are versioned and stored in a **Model Registry**.
- **Model Serving (Online):** The champion model from the registry is deployed as a **REST API endpoint** via a dedicated **AI Service** (FastAPI) or a serving tool like **KServe**. This service pulls real-time features for inference.
- **Monitoring:** Performance metrics (latency, accuracy) and data drift are monitored using tools like **Evidently AI**, triggering alerts or automatic retraining

6.5 Algorithmic Core Architecture

The Algorithmic Core is designed as a **layered, service-oriented component**:

- **Foundation Layer:** A standalone library (Python/C) providing mathematical primitives (`FUN ALGO 001, 004`).
- **Application Service Layer:** Dedicated services consuming the foundation:
  - **Search Service:** Hosts TF IDF and Levenshtein algorithms (`FUN ALGO 002`).
  - **Recommendation Service:** Hosts the hybrid recommendation engine (`FUN ALGO 003`).
  - **Caching & Sampling Service:** Manages LRU caches and sampling logic (`FUN ALGO 005`).
- **Orchestration Layer:** The **Hybrid Intelligence Layer** (`FUN ALGO 006`) that queries both custom services and external APIs, fusing results for the final user experience.

7.0 EXTERNAL INTERFACES

7.1 User Interface Specifications

7.1.1 Current MVP UI

Current ui component is flutters material ui in build ui which is customized according to need.

- **Design Philosophy:** "Functional First." The UI is clean, responsive, and navigable, but lacks a custom, branded visual identity. Colors are primarily the default Material theme (indigo primary).
- **Key Screens & Flows:**
  - **Onboarding/Auth:** Simple screens with text fields for email/password and OTP input.
  - **Course Marketplace:** A scrollable list of course cards. Each card displays title, instructor name, rating, and price.
  - **Course Player:** A basic full-screen video player using the `video_player` package with standard play/pause/seek controls.
  - **Instructor Dashboard:** A tab-based interface to view "My Courses," "Analytics," and "Create New Course."
- **Constraints:** UI elements may have inconsistent spacing or typography hierarchy as these were optimized for development speed.

7.1.2 Target Design System (Phase 2+)

A comprehensive, custom **KAAFI Design System (KDS)** will be developed to ensure consistency, brand recognition, and superior user experience.

- **Foundation:**
  - **Color Palette:** A primary brand color (to be defined), with accessible contrast ratios for text. A supporting neutral palette for backgrounds and surfaces.
  - **Typography:** A defined typography scale using a primary font (e.g., Google's **Inter** or **Poppins**) with clear roles for headlines, body, and captions.
  - **Iconography:** A consistent set of outlined icons (from libraries like **FontAwesome** or **Material Icons**).
  - **Spacing & Layout:** An 8-point grid system for margins, padding, and component sizing.
- **Components:** Reusable, styled Flutter widgets that implement the foundation:
  - We will some component using dart:UI like button,app bar or other component following design.
- **Implementation:** The design system will be packaged as a private Flutter package or a dedicated directory (`/lib/design_system/`) to be consumed across the application, ensuring a single source of truth for UI components.

7.2 Hardware & Software Interfaces

- **Hardware Interfaces:**

- **Camera:** Required for instructor identity verification during the KYC process __Phase 2¯¯. Access via `image_picker` plugin.
- **Storage:** Required for caching video content and app data. Managed via device file system APIs and Flutter's `path_provider`.
- **Network:** The application requires an active internet connection __Wi-Fi or cellular data).

- **Software Interfaces:**
  - **Operating System:** Android 8.0 __API 24¯¯and iOS 14.0□ . The Flutter framework handles OS-level abstractions.
  - **Key Libraries & SDKs:**
    - **Supabase Flutter SDK**ˇ For all data and auth operations in the MVP.But from phase 2 we will build bakend on fastapi framework.
    - **Bkash & Nagad Flutter SDKs/Plugins:** For payment processing.
    - **Video Player Plugins:** `video_player` for basic, `chewie` for a better UI, and potentially `better_player` for advanced features.

7.3 Communication Interfaces

7.3.1 API Specifications

The system exposes and consumes several APIs.

- **KAAFI Platform API __Supabase ¯¯MVP**ˇ
  - **Protocol:** REST (auto-generated by Supabase) .
  - **Authentication:** Bearer Tokens __JWT¯¯ issued by Supabase Auth.
  - **Base URL**ˇ https://uyijmaytdgepuufiboef.supabase.co
  - **Data Format:** Request and response bodies in JSON.
  - Data get by class is built for supabase.
- **KAAFI Platform API __Future Microservices □ Target):**
  - **Protocol:** RESTful HTTP/HTTPS, with potential WebSocket support for notifications.
  - **Gateway:** All requests route through the API Gateway (`https://api.kaafi.com/v1/`).
  - **Authentication:** JWT tokens validated at the Gateway.
  - **Documentation:** The API will be formally documented using the **OpenAPI 3.0 Specification __Swagger)**, providing an interactive API console.
  - **Own Backend :** We will use fast api and other tech to build our backend system.

7.3.2 Third-Party Service Integrations

Service	Integration Purpose	Protocol/Interface	Data Exchanged
Bkash Payment	Process course payments in BDT.	REST API & Mobile SDK	Payment amount, invoice ID, customer MSISDN, transaction status.
Nagad Payment	Alternative payment processing.	REST API & Mobile SDK	Merchant ID, order ID, payment amount, callback URL.
SSL Wireless SMS	Send OTP and transactional SMS.	REST API (HTTP/HTTPS	Recipient number, message text, sender ID.
SendGrid / AWS SES	Send transactional emails (receipts, notifications).	SMTP or REST API	Recipient email, subject, HTML/plaintext body.
Polygon RPC Node (via Alchemy/Infura)	Interact with blockchain for NFT minting.	JSON RPC over HTTPS	Smart contract function calls, transaction data, event logs.
Cloud Storage AWS S3 / GCP Cloud Storage)	Store and serve course video files at scale.	REST API SDK	Video files MP4, thumbnails, metadata.
Mixpanel / Firebase Analytics	Track user behavior and app analytics.	REST API SDK	Anonymized user events (e.g., `course_viewed`, `lesson_completed`).

8.0 TESTING, DEPLOYMENT & OPERATIONS

8.1 Comprehensive Testing Strategy

A multi-layered testing strategy is essential to ensure the quality, security, and reliability of the KAAFI platform across its evolving architecture and complex features.

8.1.1 Test Levels & Coverage

Testing will be conducted at all levels of the system, integrated into the CI/CD pipeline.

Test Level	Scope & Tools	Objective & Success Criteria
Unit Testing:Test smallest testable part in isolation,	Tests individual functions, classes, and widgets in isolation. **Tools:** `flutter test` Dart), `pytest` Python - future), `Jest` (if applicable).	Verify logic correctness. **Target: 80% code coverage** for critical business logic modules (e.g., payment calculation, progress tracking).
Widget Testing:Test widget using pump and other method in isolation.	Tests UI components and their interactions in the Flutter app. **Tools:** `flutter test` with `WidgetTester`.	Verify that widgets render correctly, respond to input, and update state as expected.
Integration Testing:Integrate and test things all at once.	Tests the interaction between multiple modules or with external services	Verify data flow and correct interaction between, for example, the

Test Level	Scope & Tools	Objective & Success Criteria
	(mocked). **Tools:** `integration_test` package __Flutter), `Mockito` for mocking.	Course Repository and the Supabase client.
End-to-End __E2E__ Testing:testing user workflow.	Tests complete, critical user workflows from start to finish on a real device/emulator. **Tools:** `flutter drive` or **Maestro**.	Verify core user journeys (e.g., "User can register, find a course, and complete payment").
API/Contract Testing:test api so that api contract dont break,	Tests the REST/GraphQL API endpoints for correctness and schema adherence. **Tools: Postman**, **Pact** (for contract testing between microservices).	Verify API responses, status codes, data shapes, and that changes don't break client integrations.

8.1.2 Specialized Testing

Due to the advanced nature of planned features, specialized testing protocols are required.But i need more research on this part so please ovserve or take this as initial data gather.

- **Blockchain & Smart Contract Testing:**
  - **Scope:** Smart contract logic, gas optimization, and security vulnerabilities.
  - **Tools: Hardhat** or **Truffle** test frameworks, **Slither** or **MythX** for static analysis.
  - **Process:** Deploy contracts to a local testnet __Hardhat Network) or public testnet __Polygon Mumbai). Simulate all minting, transfer, and verification functions. Conduct tests for common vulnerabilities like reentrancy and integer overflow.
- **Cryptocurrency Payment Testing:**
  - **Scope:** End-to-end payment flow using testnet cryptocurrencies (e.g., test ETH, test USDT__.
  - **Process:** Test wallet connection, transaction signing, balance checks, and confirmation listening in a sandbox environment. Validate fallback behaviors for failed transactions.
- **AI/ML Model Testing:**
  - **Scope:** Model accuracy, performance, bias, and robustness.
  - **Tools: Great Expectations** for data validation, **MLflow** for experiment tracking.
  - **Process:** Validate models against a hold-out test dataset. Monitor for prediction drift in production. Test model serving endpoints for latency and throughput.
- **Security & Penetration Testing:**
  - **Scope:** Application vulnerabilities, infrastructure misconfigurations, and data leakage.
  - **Tools: OWASP ZAP**, **SonarQube**, manual penetration testing.
  - **Schedule:** Automated scanning integrated into CI/CD. Full manual penetration test before each major release __Phases 2, 3__.

8.2 DevOps & MLOps Implementation

8.2.1 CI/CD Pipeline Design

A continuous integration and delivery (CI/CD) pipeline will be established using **GitHub Actions** (or a similar tool) to automate the software delivery process.

- **Pipeline Stages:**
  - **Lint & Format:** Run `dart format` and `flutter analyze` to ensure code style consistency.
  - **Build & Unit Test:** Compile the Flutter app for target platforms and run all unit/widget tests.
  - **Integration Test:** Run integration test suites on a dedicated emulator/simulator.
  - **Security Scan:** Run static application security testing (SAST) tools.
  - **Build Artifact:** Generate platform-specific build artifacts (APK, IPA, Web bundle).
  - **Deploy to Staging:** Automatically deploy the web build to a staging environment and mobile builds to testing distribution platforms (Firebase App Distribution, TestFlight).
  - **Deploy to Production (Manual Gate):** Require a manual approval before deploying to the live production environment.

8.2.2 Infrastructure as Code (IaC)

All cloud infrastructure for the target architecture will be defined and managed as code.

- **Tool: Terraform**.
- **Scope:** Infrastructure for microservices (compute instances, Kubernetes clusters), databases, caches (Redis), message queues, load balancers, and networking rules (VPC, security groups).
- **Benefit:** Enables reproducible, version-controlled environments. The staging and production infrastructure can be identical, eliminating "it works on my machine" problems.

8.3 Deployment & Monitoring Strategy

- **Deployment Architecture:**
  - **Environments: Development**, **Staging** (mirror of production), **Production**.
  - **Strategy: Blue-Green Deployment** for the backend microservices and web app to enable zero-downtime updates and instant rollback. Mobile app updates follow standard store release cycles.
  - **Containerization:** All backend services will be packaged as **Docker** containers for consistency.
  - **Orchestration:** Containers will be managed by a **Kubernetes** cluster in production, providing auto-scaling, self-healing, and efficient resource utilization.

- **Monitoring & Observability:**
  A comprehensive observability stack will be implemented to ensure system health and performance.
  - **Metrics Collection: Prometheus** will scrape metrics from all services, containers, and infrastructure. Key metrics: request rate, error rate, latency (p95), CPU/Memory usage, video buffer health.
  - **Logging & Tracing:** Structured logs will be aggregated using the **ELK Stack** __Elasticsearch, Logstash, Kibana) or **Loki**. Distributed tracing will be implemented with **Jaeger** or **Zipkin** to track requests across microservices.
  - **Alerting: Grafana** with **Alertmanager** will be configured to send notifications __Email, Slack, PagerDuty) when defined thresholds are breached (e.g., error rate ¯ 1% for 5 minutes, API latency ¯ 2s).
  - **Real User Monitoring __RUM¯˘ Firebase Performance Monitoring** or a similar tool will be used to track app performance metrics from actual user devices

---

9.0 RISK ASSESSMENT & MITIGATION

A proactive risk management strategy is critical for a project of KAAFI's scope and technical ambition. Risks are categorized, assessed for probability and impact, and paired with specific mitigation plans. __*Impact: H ¨High, M ¨Medium, L ¨Low)*

9.1 Technical Risks

Risks arising from the complexity, integration, and evolution of the technology stack.

Risk ID	Description	Probability	Impact	Mitigation Strategy
TEC□01	**Supabase Vendor Lock-in & Scaling Limits:** The MVP's reliance on Supabase BaaS may create migration challenges and hit platform limits as user count grows.	M	H	**1. Abstract Data Layer:** Use the Repository pattern to isolate Supabase client code. **2. Early Planning:** Begin designing the target microservice schema in Phase 2. **3. Monitor Usage:** Proactively track Supabase metrics __DB CPU, API calls) to plan migration before hitting limits.
TEC□02	**Blockchain Integration Complexity & Cost:** Smart contract bugs, high/volatile gas fees on Ethereum, or	M	H	**1. Use L2/Sidechain:** Target Polygon or other Layer 2 solutions for low, predictable

Risk ID	Description	Probability	Impact	Mitigation Strategy
	regulatory uncertainty around crypto in Bangladesh could derail Phase 3.			fees. **2. Extensive Testing:** Audit smart contracts with multiple tools and test thoroughly on testnets. **3. Regulatory Sandbox:** Engage with local fintech/blockchain communities to understand the regulatory landscape early.
TEC□03	**AI/ML Model Performance & Bias:** Recommendation or chatbot models may perform poorly, provide irrelevant suggestions, or exhibit bias, harming user trust.	M	M	**1. Start Simple:** Use rule-based or collaborative filtering as a fallback baseline. **2. Rigorous Evaluation:** Establish clear accuracy and fairness metrics before deployment. **3. Human-in-the-Loop:** Design admin dashboards to review and override poor AI suggestions.
TEC□04	**Microservices Architecture Overhead:** Premature or poorly executed decomposition into microservices can lead to operational complexity, network latency, and debugging nightmares.	M	M	**1. Start Monolithic:** Justify the MVP's monolithic design. **2. Strategic Decomposition:** Split services only when clear bounded contexts emerge (e.g., Payment Service). **3. Invest in Observability:** Implement logging, tracing, and monitoring from the start of Phase 2 (e.g., with Prometheus/Grafana).

9.2 Business & Market Risks

Risks related to market acceptance, competition, and the business model.

Risk ID	Description	Probability	Impact	Mitigation Strategy
BUS□01	**Low Instructor Adoption:** Failure to attract a critical mass of qualified, verified instructors to create high-quality content, resulting in an empty marketplace.	H	H	**1. Streamline Onboarding:** Make the verification and course creation process extremely simple. **2. Incentivize Early Creators:** Launch with a limited-time, zero-commission period or creator grants. **3. Partner with Institutions:** Collaborate with vocational training centers or industry associations to source initial instructors.
BUS□02	**Learner Price Sensitivity & Trust:** The target audience may be unwilling to pay even 1000 BDT for online micro-courses, or may not trust the platform's payment/quality guarantees.	M	H	**1. Free Tier:** Offer a significant number of free introductory courses to build trust and habit. **2. Social Proof:** Highlight ratings, reviews, and learner success stories prominently. **3. Money-Back Guarantee:** Implement a clear, no-questions-asked refund policy for a short period (e.g., 7 days).
BUS□03	**Intense Local & Global Competition:** Existing platforms (e.g., 10 Minute School, Udemy) may replicate features or use their scale to dominate the market.	H	M	**1. Unique Value Proposition (UVP):** Double down on **verification** and **practical, local skills** as key differentiators. **2. Niche Focus:** Initially dominate a specific vertical (e.g., "home maintenance skills") before expanding. **3. Community Building:** Foster strong instructor-learner interactions (Q&A, forums) to increase platform stickiness.
BUS□04	**Regulatory Hurdles for EdTech & FinTech:** New or unclear regulations regarding online education content, digital	M	M	**1. Legal Consultation:** Seek basic legal advice early in Phase 2 regarding Terms of Service and Privacy Policy. **2. Design for Privacy:** Implement data minimization

Risk ID	Description	Probability	Impact	Mitigation Strategy
	payments, or data privacy could impose compliance burdens.			and clear user consent flows from the start __GDPR-inspired principles). **3. Stay Informed:** Monitor announcements from Bangladesh Bank and BTRC related to digital services.

9.3 Project Management Risks

Risks related to the execution of the project plan, team, and timeline.

Risk ID	Description	Probability	Impact	Mitigation Strategy
PM□01	**Scope Creep in Advanced Phases:** The allure of features like Blockchain and AI may lead to uncontrolled expansion of Phase 3/4 scope, delaying the core viable product.	H	M	**1. Phase-Gated Development:** Treat each phase as a contract. No work on Phase 3 begins until Phase 2 is **signed off as complete**. **2. Maintain a Backlog:** Park all "nice-to-have" ideas in a future backlog, to be re-prioritized at the start of the next phase.
PM□02	**Skill Gaps in Advanced Technologies:** The development team (or solo developer) may lack expertise in blockchain, DevOps, or ML, leading to delays or poor implementation.	M	H	**1. Phased Learning:** Schedule dedicated research/learning sprints at the start of each technically new phase. **2. Leverage Managed Services:** Use services like Alchemy (blockchain), Vertex AI __ML¯¯, etc., to reduce foundational complexity. **3. Contingency Budget:** Plan for the potential cost of consulting an expert for a short review.
PM□03	**Underestimation of Maintenance & DevOps Load:** The ongoing effort required for server maintenance, monitoring, and updates for a scaled platform is underestimated, draining resources.	M	M	**1. Automate Early:** Invest in CI/CD and Infrastructure as Code __Terraform) in Phase 2 to reduce manual toil. **2. Budget for Operations:** Include cloud hosting and monitoring service costs in the long-term financial plan from the start.

9.4 Risk Mitigation Strategies (Summary & Ownership)

- **Primary Strategy: The Hybrid SDLC Model.** The chosen methodology __Section 4.1¯¯ is itself a risk mitigation tactic. The **Spiral model elements** force explicit risk identification and prototyping before full commitment.
- **Ownership:** Its an academic project so the primary ownership goes to me.
- **Monitoring:** The **Risk Register __Appendix F¯¯** will be a living document. Risks will be reviewed and updated at the conclusion of each major phase.

10.0 PROJECT MANAGEMENT

This section outlines the plan for executing the KAAFI project, aligning the phased roadmap with concrete milestones, resources, and measurable success criteria.

10.1 Project time line

Based on course offered by university i will reframe and learn specfice part and implement.

10.2 Resource Allocation

The project is initially executed as a solo or small-team academic endeavor, with resource needs evolving per phase.

Resource Type	Phase 12_ MVP & Scale)	Phase 3_ Advanced & Beyond)
Human Resources	**Primary Developer (You):** Full-stack Flutter/Dart, basic DevOps.**Supervisor:** Guidance, requirement validation.	**Expanded Team __Potential):** May require specialized skills: Blockchain/Solidity Developer (consultant/partner). ML Engineer (or use of managed AI APIs).
Software & Tools	IDEˇ VS Code / Android Studio. Version Control: GitHub. Backend: Supabase __Free Tier) Design: Figma (for UI mockups).	CI/CDˇ GitHub Actions __Free). Infrastructure: Terraform. Blockchain: Hardhat, Alchemy/Infura __Free Tiers). MLˇ Google Colab, MLflow.
Hardware	Desktop	May require more powerful machine for ML model training or running local blockchain nodes.
Financial Budget	Minimal (domain name, potential low-tier cloud services).	Increased for:Cloud hosting costs __AWS/GCP credits). Blockchain transaction (gas) fees on mainnet. Potential costs for security audits or legal consultation.

10.3 Success Criteria & Acceptance Plan

Success is measured against specific, measurable criteria aligned with the goals of each phase.

Phase 1 __MVP¯¯ Acceptance Criteria:

- **Functional:** The three core user journeys must be demoable without critical bugs:

- Learner can open app .
- enroll course,add to wish list,add to fav list.
- learner can discuss with ai the course related work.
- *Learner* can watch the first lesson of the enrolled course.

- **Technical:** The codebase must pass a basic quality gate (e.g., no `analysis_options` linter errors, core widgets have tests).

Phase 2 __Securing,Developing and Hardening critical parts) □ Success Criteria:

- **Performance:** Application meets NFRs (e.g., home screen load ¯2s on 4G, tested via profiling).
- **Process:** Full CI/CD pipeline is operational; every code commit triggers automated build and test.
- **Quality:** Achieve □80% unit test coverage for critical business logic modules.
- **Security:** Resolve all critical/high vulnerabilities identified in the security audit.

Phase 3 __Advance integration) □ Success Criteria:

ML Success Criteria:

- **Deployment of at least one production ML model** that moves beyond the MVP's generic API integration to provide data-driven personalization or automation.
- **Measurable Performance Improvement:** The model must demonstrate a clear, positive impact on a key business metric (e.g., engagement, retention, content quality) against a defined baseline, as per the initiatives above.
- **MLOps Foundation:** Establishment of a basic pipeline for model training, evaluation, and versioning, ensuring the model is not just a one-off experiment but a maintainable asset.
- **Algorithmic Integration:** The foundational library (`FUN□ALGO□001`) must be successfully called by at least one application service (e.g., the Recommendation Service `FUN□ALGO□003`). Performance and correctness must be validated against a benchmark.

Acceptance Plan:

- My plan is to build it till final year project whatever i learn i will integrate from Algorithm to Cryptography with block chain.

11.0 CONCLUSION

This Software Requirements Specification __SRS¯¯ document provides a comprehensive, phased blueprint for the KAAFI Verified Micro-Skills Platform. It serves as the definitive **single source of truth** that aligns all stakeholders from the academic supervisor and development team to future users on the project's vision, requirements, and roadmap.

By detailing the **completed Minimum Viable Product (MVP)** alongside the strategic **Four-Phase Implementation Roadmap**, :

- **As a Development Guide:** Its development plan for entire semsiter.
- **As a Project Plan:** It defines the scope, timeline, and success criteria for the academic project, establishing a clear contract for deliverables.
- **As a Living Document:** It acknowledges the evolutionary nature of the KAAFI project, with a structure designed to be updated as the project progresses through each phase, ensuring the document remains relevant.

12.0 REFERENCES

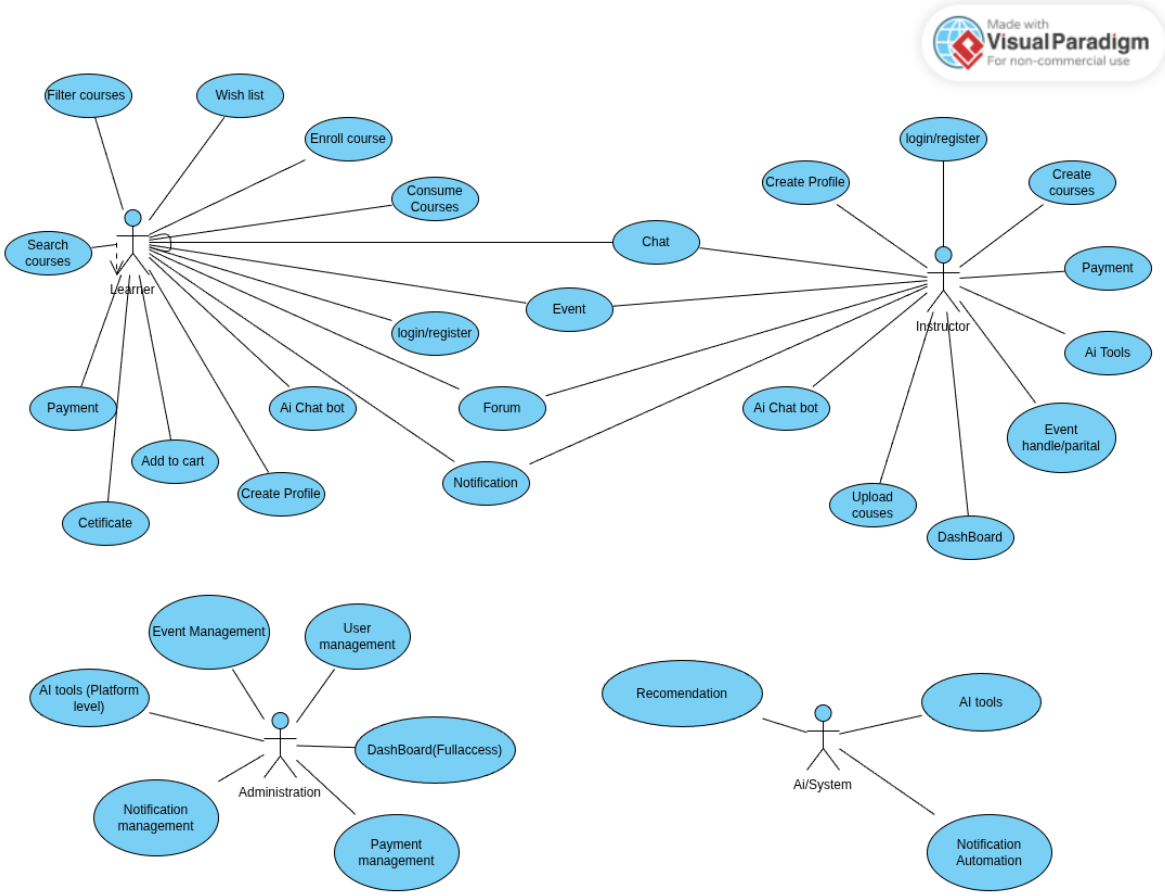
All external documents, standards, and key sources referenced in this SRS are listed here.

- IEEE. *IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications*. IEEE Computer Society, 1998.
- ISO/IEC/IEEE. *ISO/IEC/IEEE 29148:2018, Systems and software engineering — Life cycle processes — Requirements engineering*. 2018.
- Sommerville, Ian. *Software Engineering, 10th Edition*. Pearson, 2015. (Core textbook for software requirements and lifecycle principles).
- Project Lab II Guidelines. Institute of Information Technology, University of Dhaka. (Refer to your specific course handout or document).
- Flutter Documentation. https://flutter.dev (Accessed January 2026).
- Supabase Documentation. https://supabase.com/docs (Accessed January 2026).
- Perforce. "How to Write a Software Requirements Specification (SRS) Document." *Perforce Blog*, 11 July 2025.

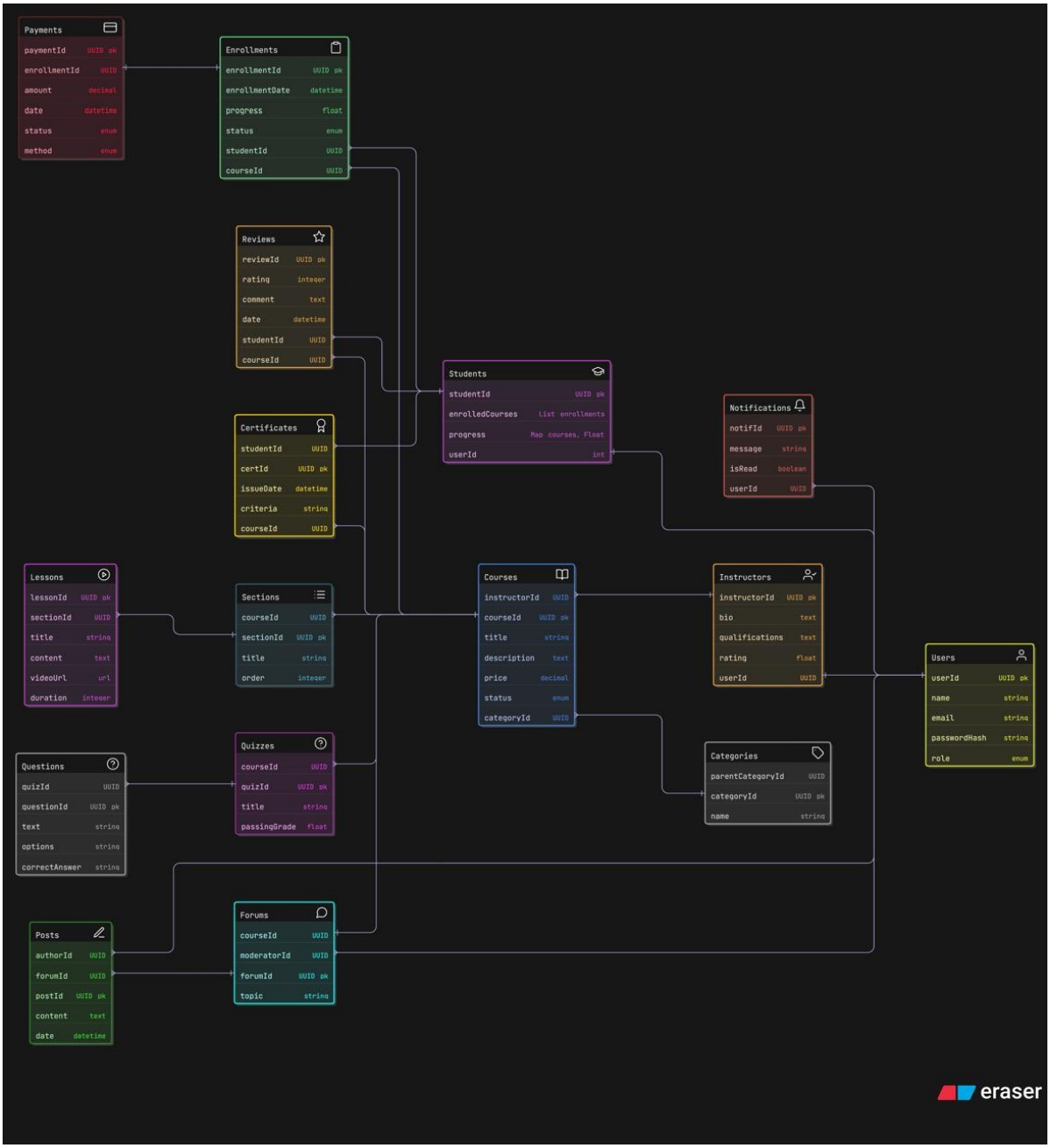
APPENDICES

The appendices contain supporting details that are critical for implementation but too granular for the main body of the SRS.

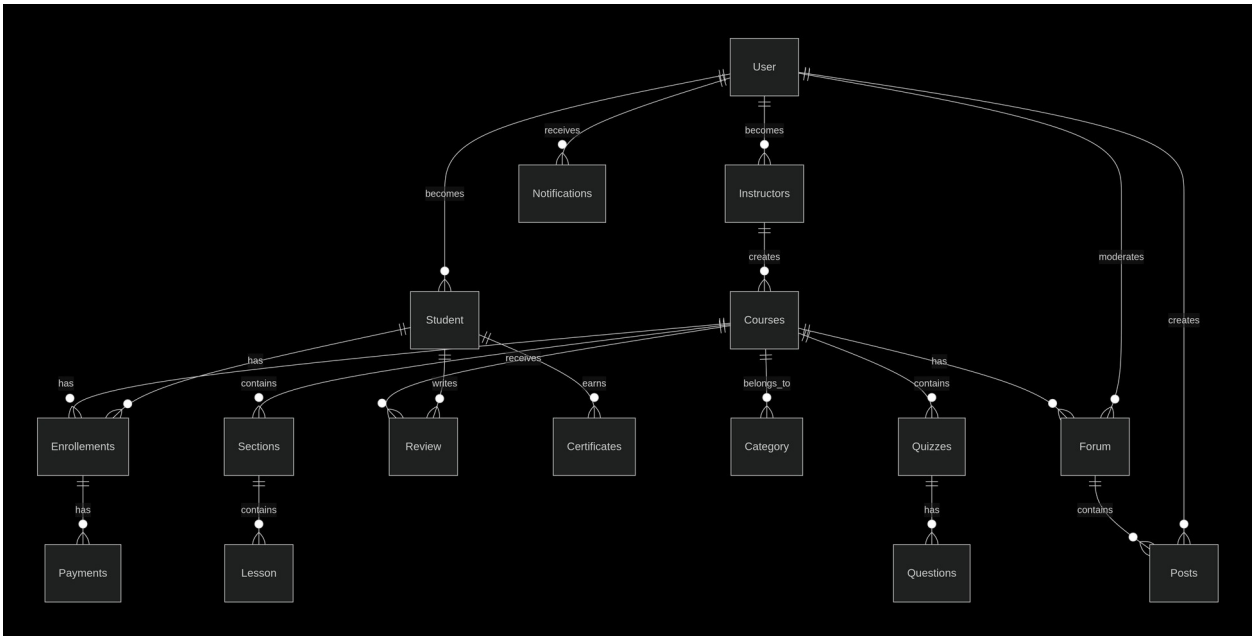
Appendix A:Use Case Diagram



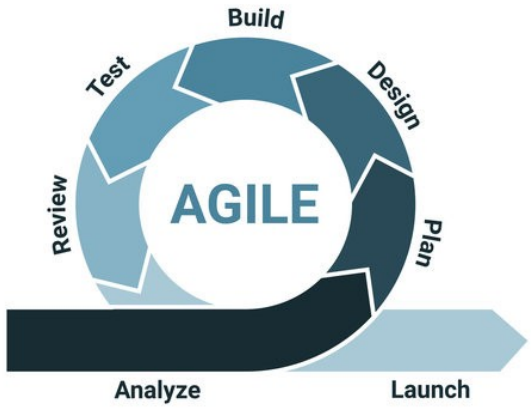
Appendix B: Class Diagram

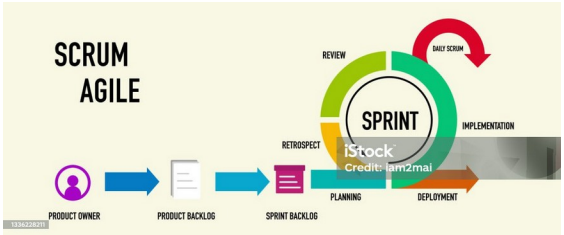


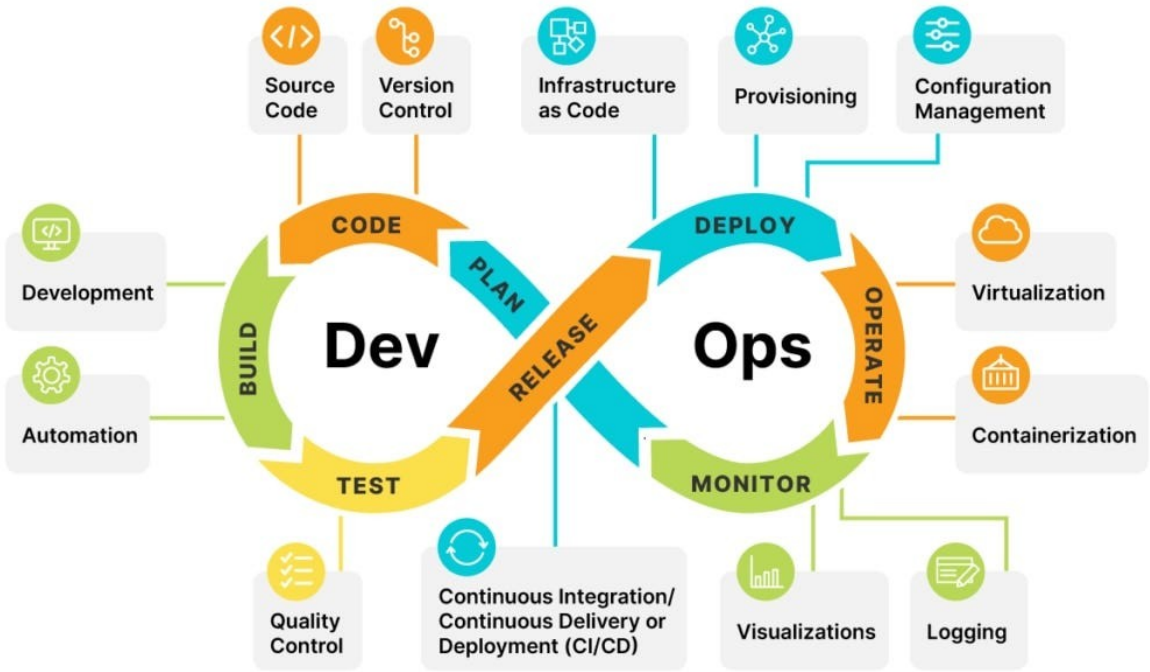
Appendix C: Database ERD



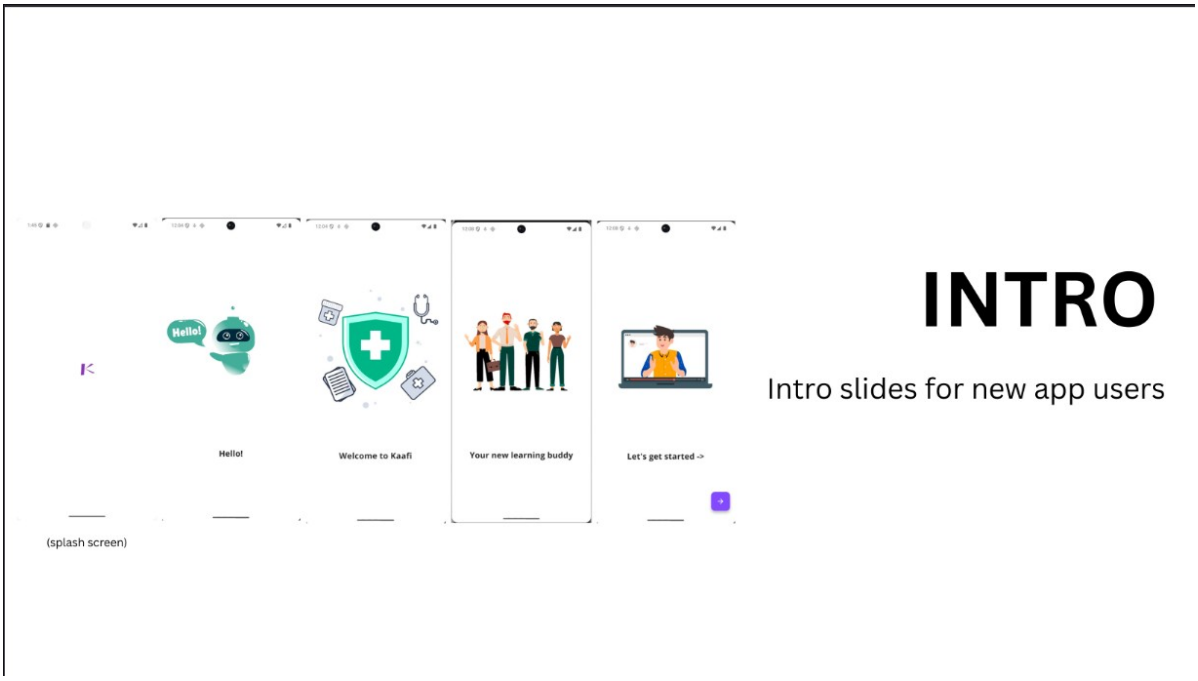
Appendix C.1:SDLC Images

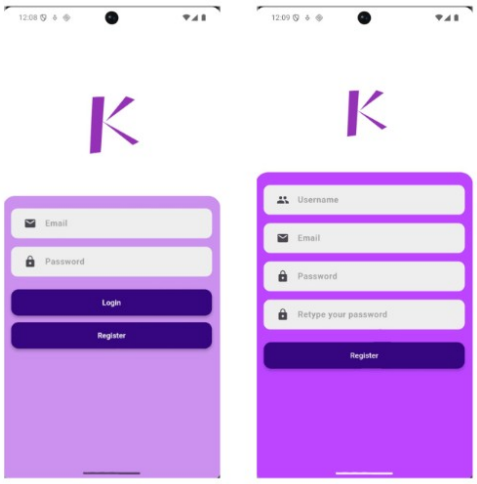






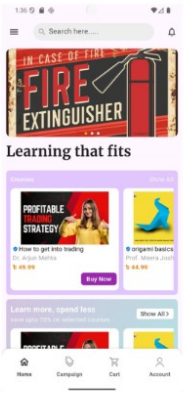
Appendix D: UI/UX Mockups & Prototypes

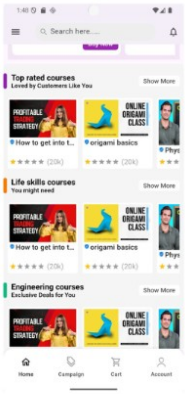




Auth Screens

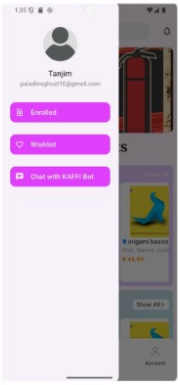
User login and User Registration forms, Users can register with emails.





(Home Screen)

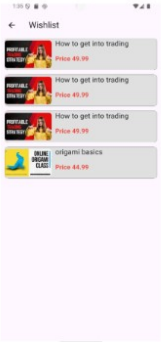
Homescreen Includes banner of featured courses, Carousel and catagorized courses



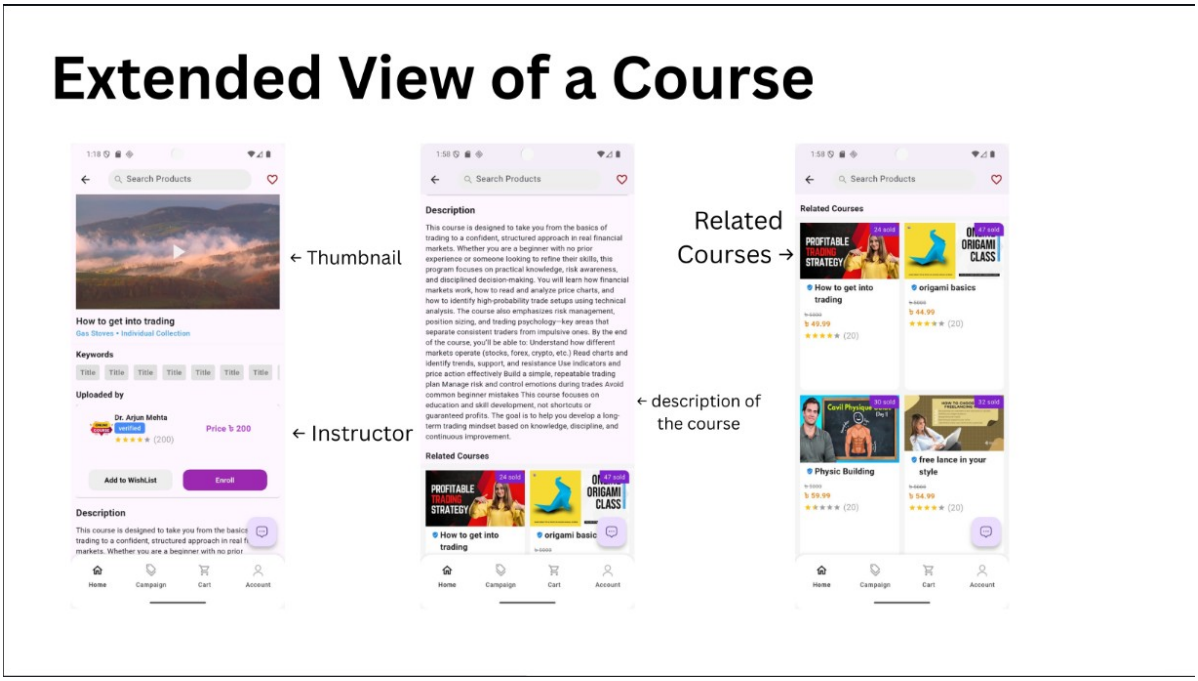
(Drawer)

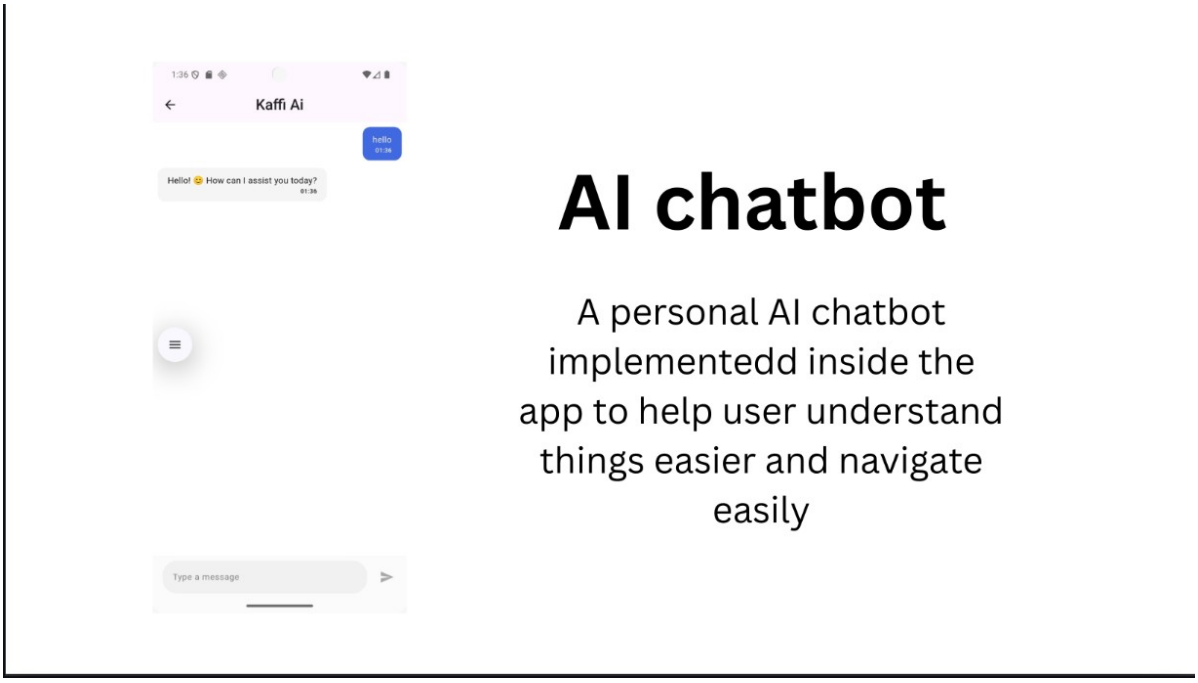
Shows username, email, and 3 buttons for
-Enrolled Courses
-Wishlist
-KAAFI Bot

(Enrolled Courses)

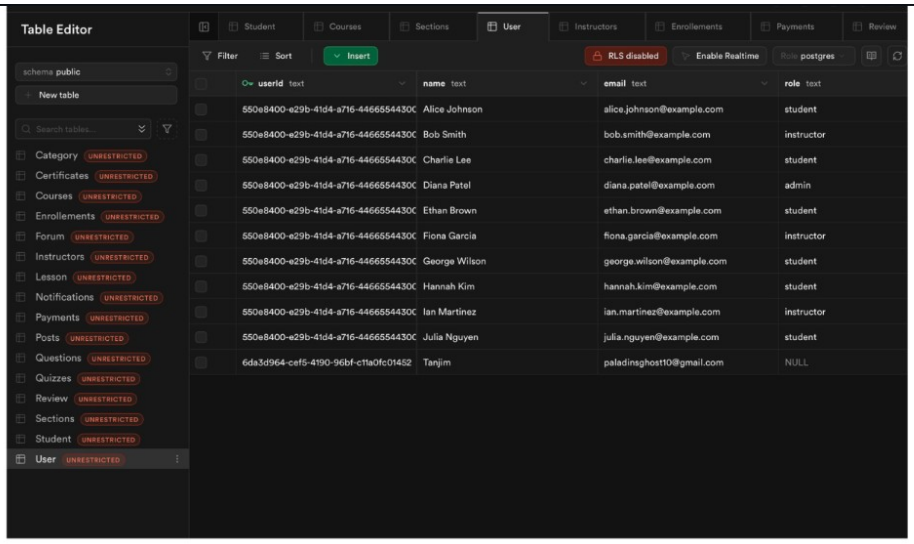


(Courses in wishlist)

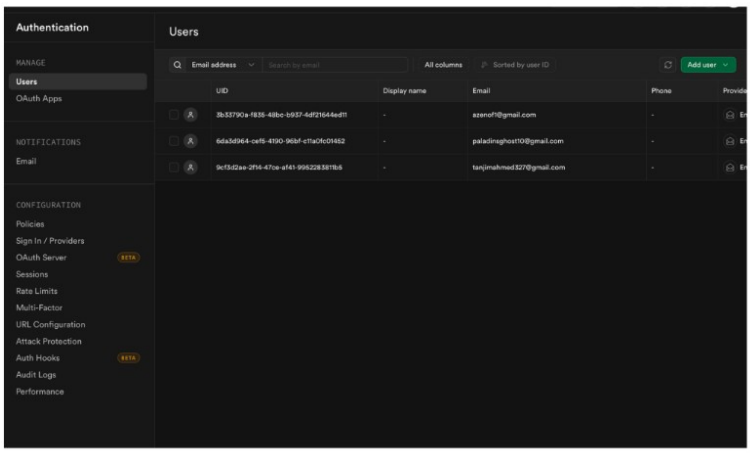




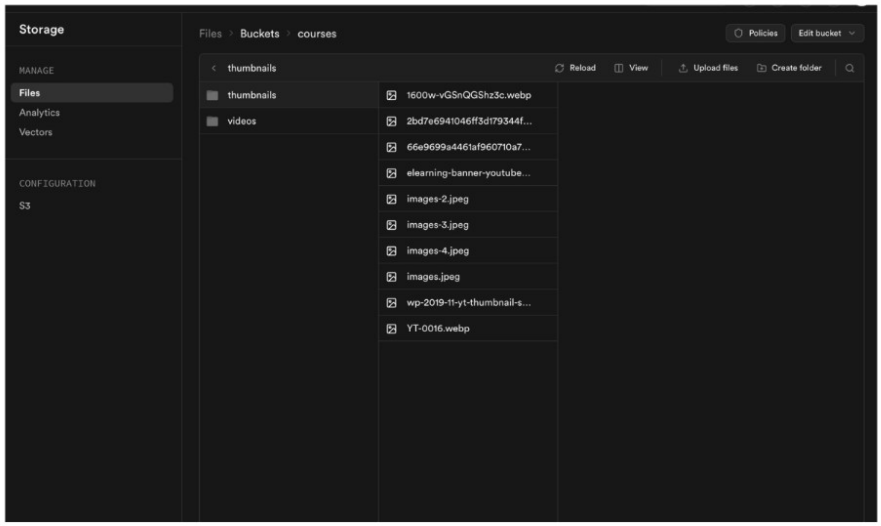
Appendix E: Supabase backend



A Database to store all the data In real time and use it as needed



Storing authenticated user data



uploaded data is stored here, and then you are least created to connect them with the table so that the users can click on a button which will take them to the video using the URL